DataAnalysis

May 13, 2026

1 Current progress

- ☒ Selection of relevant data
- ☒ Data extraction
- ☒ Processing of initial data
- ☒ Data normalization
- ☒ Initial visualization
- ☒ Initial commentary
- ☒ (Optional) Notebook to PDF automation
  - Action repository
- ☒ (Optional) Marimo toy example
  - Interactive notebook app
- ☐ (Optional) Power BI dashboard
  - Blocked from publishing due to Microsoft's focus on enterprise
  - See screenshot for initial experimentation
- ☐ Technical documentation (in progress)
- ☐ Formulation of research questions (in progress)
- ☐ Selection of statistical tools and methods (in pogress)
- ☐ Statistical analysis
- ☐ Final chart generation
- ☐ Analysis write-up
- ☐ Conclusion write-up

2 A stream-of-consciousness walkthrough of the project to date

2.1 Hvilke turistdata har jeg specifikt brug for?

Projektet startede ud med ét hovedspørgsmål: Hvad for nogle interessante opdagelser kunne jeg lave ved at analysere turistdata fra Danmark Statistiks hjemmeside? (Den samme side findes også hos VisitDenmark.) Til dette formål ville jeg også gøre brug af andre datasets, for turistdatene i sig selv ikke var fyldestgørende til mit formål.

OBS: Turistdataene handler specifikt om overnatninger, men i selve koden navngiver jeg dem som `visits`.

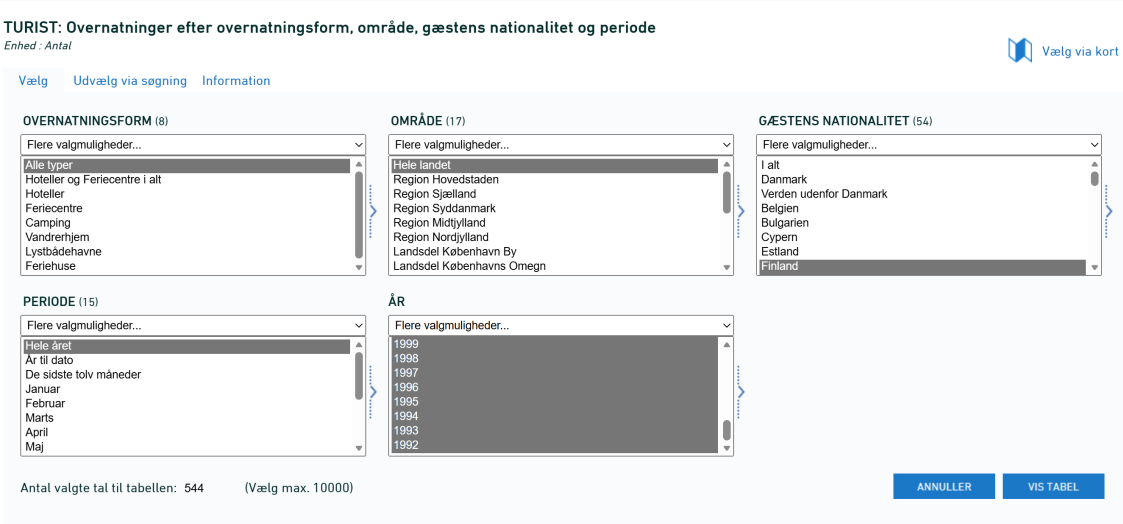
Jeg begyndte at lege med instillingsmuligherne for at se, hvilke typer uddrag gav mest mening eller gav anledning til underspørgsmål, som jeg ville undersøge nærmere. Felterne på hjemmesiden hed følgende:

- Overnatningsform (fx hotel, feriecenter)

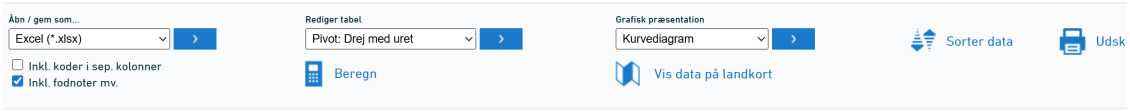
- Område (region eller landsdel indenfor Danmark, hvor overnatningen fandt sted)
- Gæstens nationalitet
- Periode (fx en specifik måned eller hele året)
- År

Ved at vælge flere muligheder kan kombinationerne hurtigt blive rigtig store, og man må maks. generere et uddrag på 10.000 tal ad gangen. Heldigvis havde jeg ikke brug for særlig mange, fordi de spørgsmål, jeg syntes var mest interessante at undersøge, drejede sig om årlige trends på tværs af landet som en helhed. Dvs. at kategorier såsom område og overnatningsform var fuldstændig irrelevante.

Det uddrag, som jeg fik lavet, ser sådan her ud:



Og gav følgende tabel (til og med 2025):



Overnatninger efter periode, område, overnatningsform, gæstens nationalitet og tid

	1992	1993	1994	1995	1996	1997	1998	1999	2000	2001	2002	2003
Hele året												
Hele landet												
Alle typer												
Finland	177 317	119 236	125 899	119 348	128 554	132 048	124 549	132 740	156 845	147 957	139 090	140 125
Frankrig	138 251	129 612	124 791	119 187	115 237	132 280	118 533	109 737	112 375	115 779	123 252	125 359
Nederlandene	1 275 478	988 851	1 044 004	1 171 302	1 121 585	1 087 017	1 006 153	968 978	946 526	970 897	956 839	1 101 480
Irland	0	0	0	0	0	0	0	0	1 012	846	547	624
Island	56	33	28	24	62	51	18	3	3 533	2 714	3 062	5 151
Italien	213 172	167 243	180 496	164 349	200 305	195 137	196 669	194 860	196 395	192 671	187 694	195 500
Norge	1 557 092	1 651 075	1 661 645	1 482 748	1 707 104	1 884 710	1 990 024	1 952 288	2 091 732	2 055 563	2 300 662	2 394 809
Portugal	6	0	0	61	100	6	0	21	1 413	776	1 427	610
Schweiz	36 074	33 158	31 854	34 666	89 038	78 638	80 991	82 453	91 877	96 401	101 065	92 031
Spanien	3	3	6	3	72 127	80 856	64 168	59 234	74 721	66 912	79 224	77 711
Storbritannien	368 968	361 297	407 339	389 924	384 067	383 426	380 647	413 441	493 159	561 812	530 452	497 576
Sverige	3 038 262	2 276 078	2 193 737	1 955 260	2 086 984	2 220 779	2 183 957	2 284 986	2 577 709	2 309 639	2 243 705	2 233 658
Tyskland	18 128 012	19 814 481	19 351 882	19 794 759	19 199 514	18 443 939	17 882 341	16 112 795	15 665 787	15 001 762	15 111 464	15 342 041
Østrig	224	97	102	243	33 718	32 473	33 113	36 094	33 508	35 615	31 077	29 545
Canada	0	0	0	0	13 132	14 441	14 433	19 065	29 650	35 916	36 512	33 858
USA	310 229	296 629	309 796	289 083	276 846	282 177	297 597	292 573	335 915	348 320	308 625	312 033

Grunden til, at jeg valgte sådan et uddrag, var fordi at jeg havde begyndt at tænke på spørgsmål, som jeg selv vil kunne svare vha. et større overblik fremfor månedlige tal. Der var bl.a. et spørgsmål om, hvordan forskellige geopolistike begivenheder har påvirket rejsevaner til Danmark fra flere lande. Jeg tænkte fx på Brexit og legede med idéen om, at Storbrittaniens forladelse af EU måske kunne have haft en nedtrykkende effekt på deres rejser til lande indenfor EU, som jo også ville påvirke Danmark.

(Det viste sig nok ikke at være tilfældet ud fra de foreløbige regningner, som har jeg lavet indtil videre. Det kan måske dels skyldes, at det nok er rigere mennesker fra landet, som hovedsageligt rejser til Danmark. Og pga. stigende økonomisk ulighed har deres formue ikke været påvirket af Brexits katastrofale indtryk på det britiske økonomi i nær så høj grad som andre demografier i landet.)

For at kunne svar på sådan et spørgsmål blev jeg jo nødt til at sammenligne landet med andre lignende økonomier. Frankrig sammenlignes fx ofte med Storbrittanien.

Og så er der COVID, hvor man kan spørge om, hvorvidt den har haft en længerevarende effekt på rejser generelt. Under alle omstændigheder havde jeg i hvert fald en retning at gå imod, men der skulle lidt mere til, før jeg kunne blive klogere på noget.

(Det viste sig i øvrigt at jeg fandt på langt bedre spørgsmål end ovenstående, men dette er først og fremmest en kronologisk fortælling af min tankegang.)

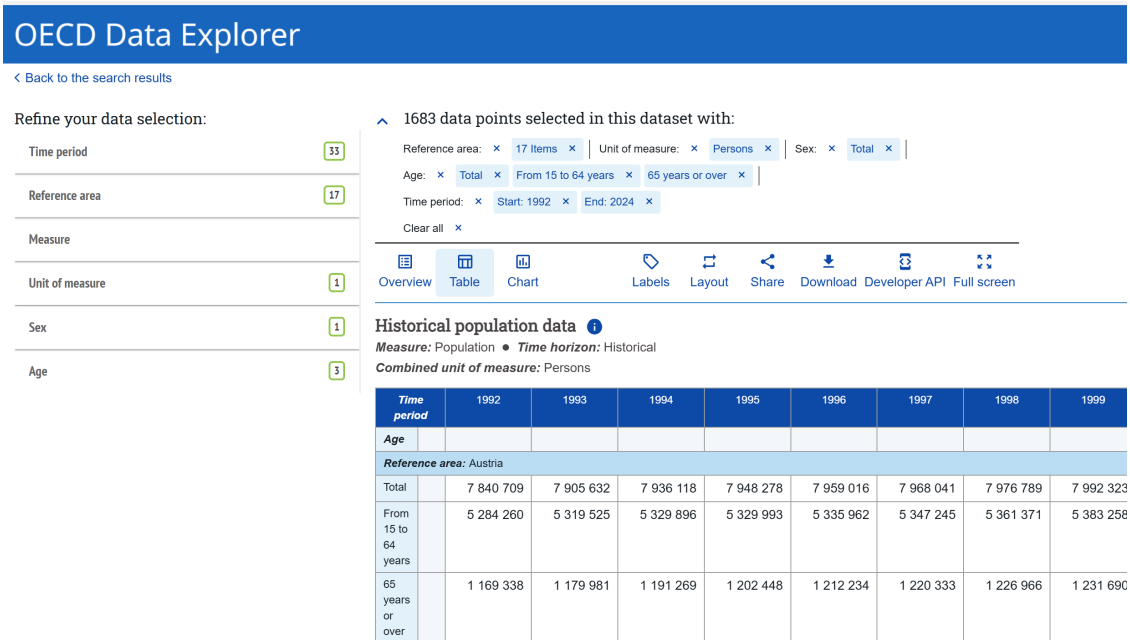
2.2 Hvad mere har jeg brug for?

Turistdataene kunne jo kun vise overnatninger, men de siger slet ikke noget om, hvor mange de er ift. landenes indbyggertal. Jeg blev først nødt til at kunne skalere overnatninger til befolkningens størrelse, før jeg kunne komme nogen vegne. Heldigvis findes der en del open source data, som sammenlægger sådanne nogle tal og har ovenikøbet højere granularitet, hvor befolkningstallene fx kan opdeles efter aldersgruppe og køn.

Siden hedder OECD Data Explorer.

(Den side som jeg linker til ovenfor viser en søgning om "dependency ratio" i Danmark, dvs. procentdelen af landet som er mellem 20 og 65. Jeg ønsker, jeg havde fandt det noget før, for det var lige præcis én af de statiskker, som jeg ville regne ud på en anden måde. Det beskriver jeg også nærmere nedenfor. Der er dog god grund til, at jeg først lægger mærke til det nu, for man må ikke vælge det som "measure" samtidig med de andre felter, jeg havde valgt.)

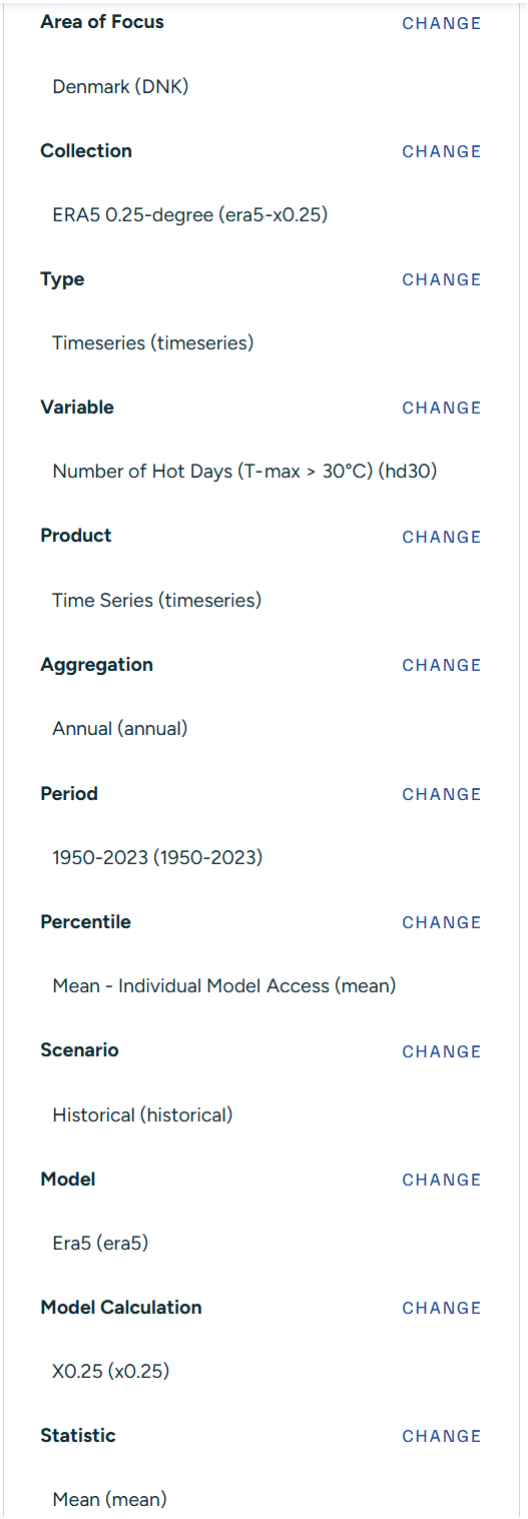
Det specifikke uddrag, som jeg fik lavet, ser sådan her ud (til og med 2024):



Jeg var dog ikke færdig endnu. Min kæreste foreslog, at jeg kunne undersøge om klimaforandringer har haft nogen effekt på turister fra det sydlige Europa, eller i hvert fald om lande, som har oplevet mere varme og hede, har i forholdsvis højere grad valgt at rejse til Danmark. Dér ville lidt mere granularitet være nyttig, som fx månedlige tal. Men omfanget for projektet skulle jo begrænses. Jeg valgte alligevel at kigge på data, som jeg kunne finde til det formål, og jeg fandt Climate Change Knowledge Portal.

Deres API var virkelig nemt at anvende, så jeg tog et uddrag for alle de lande som jeg ville have i mit dataset, hvor jeg fik det gennemsnitlige antal dage om året med over 30 graders varme. De oprindelige data måles per time og lægges sammen, og derfor er tallene ikke heltal.

Konfigurationen pr. land ser sådan ud (til og med 2023), og der kan ses et eksempel API-kald ved at klikke på linket:



2.3 Valg af dataframe-library

Nu hvor jeg havde lavet alle de nødvendige uddrag (min lille "ekstrakt-del" til en forestillet ETL-pipeline), kunne jeg endelig begynde at kode og lave nogle indsigter. Men først skulle tabellene

transformeres fra rå data til en mere brugbar form. Nu kommer jeg til den lidt mere tekniske del og kommer dermed også til at slå over i engelsk for både at være så præcis som muligt, og for at kunne skrive lidt hurtigere, selvom jeg nu også tænker på dansk samtidig i baghovedet.

Thankfully, all the sources I used had well-formatted and complete data, so there wasn't much cleaning and processing I needed to do there. Moreover, I could simply download a full .csv for both the tourist data and the population data. The JSON data for the climate numbers was also trivial to transform into a .csv format. The issue is, I had a big decision to make before I took all of this from a file on disk to some representation in memory. The most obvious library that came to mind to use for this was of course pandas, as from my education in data science from 7 years ago, this was the gold standard at the time.

However, I was never a fan of the API. There are a lot of quirks about the library that make it difficult to quickly iterate. In short, the code is easy enough to read and understand after writing it, but figuring out which specific approach to take can be a huge timesink for a project. Again here I was lucky, as I decided very early on to see if there had been any recent developments in this space, and indeed there had! Besides new interoperability libraries like Narwhals, the "new kid on the block" that has matured enough in the last 3 years to be production-ready is Polars.

It seems to be extremely liked by people who use it - "come for the speed, stay for the API" - and it can very easily convert to and from a pandas-formatted dataframe. It seems relatively feature complete, extremely quick, well-documented and supported, easy to pick up, and the API one of the most intuitive things I have come across. There are a lot more beneifts I could mention - e.g. how many performance benefits you get for free and how easy it is to squeeze out even more performance if necessary - but in short, it is very much worth the investment and the learning curve to use the library. It seems to be the current best contender for a more future-proof and well-designed dataframe library.

In any case, I was very quickly up and running with processing the data into a more appropriate format, keeping in mind that I was learning an entirely new library from scratch for this. The tourist data was small enough to just edit manually - a few nice VS Code tricks made it quick and easy to bulk delete the empty fields at the beginning of each line - and the climate data is downloaded automatically in an appropriate format through the code I've written. The only file that needed some processing was the population data, as there were too many columns that I didn't need. So without further ado, let's look at some code!

(The code in this notebook is essentially a copy-paste of the scripts I wrote in the scripts folder, but edited for a notebook format. I also leave out non-essential code that doesn't contribute to the final product - basically the rudimentary tests that I created in 2_sanity_checks.py.)

2.4 Indlæsning og bearbejdning af datafilerne

Let's start with looking at the first bit of code that needs to be fun, after which I'll explain some quirks that I otherwise have as comments in the original script files. There might be additional relevant commentary that I have as well along the way.

NB: The code for this section comes from 1_create_files.py.

[1]:
```
import polars as pl
import requests
import json
```

```
import csv
```

[2]:
```
# List of tuples to more easily fit with what pl.read_csv() expects
POPULATION_COLS = [
    ('REF_AREA', 'iso_code'),        # Three letter ISO code for a country
    ('Reference area', 'country'),
    ('AGE', 'demo'),                 # Short for "demographic"
    ('TIME_PERIOD', 'year'),
    ('OBS_VALUE', 'demo_total'),
]

pop = pl.read_csv(
    source='../data/population-raw.csv',
    columns=[col[0] for col in POPULATION_COLS],
    new_columns=[col[1] for col in POPULATION_COLS],
    schema_overrides={'OBS_VALUE': pl.Float64}, # Avoid parsing errors
)

print(pop)
```

```
shape: (1_683, 5)

 iso_code   country          demo     year   demo_total
 ---        ---              ---      ---    ---
 str        str              str      i64    f64

 AUT        Austria          _T       1992   7.840709e6
 AUT        Austria          _T       1993   7.905632e6
 AUT        Austria          _T       1994   7.936118e6
 AUT        Austria          _T       1995   7.948278e6
 AUT        Austria          _T       1996   7.959016e6
 …          …                …        …      …
 GBR        United Kingdom   Y_GE65   2023   1.2736274e7
 GBR        United Kingdom   Y15T64   2023   4.3259738e7
 USA        United States    Y15T64   2023   2.16168053e8
 USA        United States    _T       2023   3.34914895e8
 USA        United States    Y_GE65   2023   5.9248361e7
```

NB: I wrap expressions in print statements even though it's not necessary in a notebook, so that they can be copy-pasted directly into a script without having to be modified.

Much like with pandas, Polars has a very easy way of reading in .csv files. I don't need to specify the `source` as a keyword argument, but for visual consistency, it's nicer to include. The `POPULATION_COLS` list defines the column names that I want to read in (i.e. excluding the rest), as well as and the replacement column names that I want use for the dataframe itself. Due to the function expecting a list for both `columns` and `new_columns`, the best approach is a list of tuples that associates the old column names with the new, and then a list comprehension to extract one or the other set of values.

The reason for the `schema_overrides` is that, although population numbers should be integers, a few entries (in particular for Portugal) have a .5 on the end of their values, and Polars has a chance of giving parsing errors if it has read in a bunch of what it thinks to be integers and then encounters a non-integer (e.g. a float or a text string).

Let's actually look at the error we get if we don't include this argument:

[3]:
```
try:
    pop_error = pl.read_csv(
        source='../data/population-raw.csv',
        columns=[col[0] for col in POPULATION_COLS],
        new_columns=[col[1] for col in POPULATION_COLS],
        schema_overrides={'OBS_VALUE': pl.Int64}, # To ensure consistent␣
 ↪behavior
    )
except Exception as error:
    print(type(error))
    print(error.args)
    print(error)
```

```
<class 'polars.exceptions.ComputeError'>
("could not parse `9952493.5` as dtype `i64` at column 'OBS_VALUE' (column
number 19)\n\nThe current offset in the file is 2315 bytes.\n\nYou might want to
try:\n- increasing `infer_schema_length` (e.g. `infer_schema_length=10000`),\n-
specifying correct dtype with the `schema_overrides` argument\n- setting
`ignore_errors` to `True`,\n- adding `9952493.5` to the `null_values`
list.\n\nOriginal error: ```invalid primitive value found during CSV
parsing```",)
could not parse `9952493.5` as dtype `i64` at column 'OBS_VALUE' (column number
19)

The current offset in the file is 2315 bytes.

You might want to try:
- increasing `infer_schema_length` (e.g. `infer_schema_length=10000`),
- specifying correct dtype with the `schema_overrides` argument
- setting `ignore_errors` to `True`,
- adding `9952493.5` to the `null_values` list.

Original error: ```invalid primitive value found during CSV parsing```
```

I encountered the error due to the default `infer_schema_length` being 100, so the first 100 values are used by Polars to guess what type it should use. If you have a float after that, it could cause the above problem. Hence, I just tell Polars ahead of time to expect the column to all be floats. We then deal with this problem by casting everything to ints after the fact. This can't be done when reading in the data.

(On a side note, these sorts of error messages are quite nice, in that they give suggestions for how to fix the problem. I recommend against using `ignore_errors` in most cases though, as those errors

will be read in as null values.)

Moreover, the reason for doing things this way is that it's the quickest solution to ensuring the values are consisent, without editing the raw file, which allows for reproducibility. A different approach could instead have been to programmatically remove the decimal portion of such numbers, but I preferred this tactic.

[4]:
```
fixed_demo_count = pl.Series(
    'demo_total',
    pop.select(pl.col('demo_total').cast(pl.Int64))
)
pop.replace_column(pop.get_column_index('demo_total'), fixed_demo_count)
# Print instead of writing to disk in the notebook
# Notice the new datatype for demo_total in the output
print(pop)
# pop.write_csv('../data/population-processed.csv')

# Use these codes to manually edit tourists-processed.csv
COUNTRY_ISO_CODES = sorted(pop.unique(subset='iso_code').get_column('iso_code').
 ↪to_list())
print(COUNTRY_ISO_CODES)
```

```
shape: (1_683, 5)

 iso_code   country          demo     year   demo_total
 ---        ---              ---      ---    ---
 str        str              str      i64    i64

 AUT        Austria          _T       1992   7840709
 AUT        Austria          _T       1993   7905632
 AUT        Austria          _T       1994   7936118
 AUT        Austria          _T       1995   7948278
 AUT        Austria          _T       1996   7959016
 …          …                …        …      …
 GBR        United Kingdom   Y_GE65   2023   12736274
 GBR        United Kingdom   Y15T64   2023   43259738
 USA        United States    Y15T64   2023   216168053
 USA        United States    _T       2023   334914895
 USA        United States    Y_GE65   2023   59248361

['AUT', 'BEL', 'CAN', 'CHE', 'DEU', 'ESP', 'FIN', 'FRA', 'GBR', 'IRL', 'ISL',
'ITA', 'NLD', 'NOR', 'PRT', 'SWE', 'USA']
```

There's not a shorter idiomatic way to replace a column, as Polars often expects you to simply make a copy of the column with the changes you want to apply. However, in this case it makes more sense for us to entirely replace it. This can be done by creating a series (same basic idea as in pandas) and replacing that column with our newly converted values.

Finally, let's also grab the unique country ISO codes, as these will function as the the informal

index values for all our dataframes. Polars doesn't use indices for its dataframes, unlike pandas, and honestly it is better for it.

[5]:
```
import io # for StringIO to create a dummy file handle

# The dataset ends in 2023
# The year consistently uses July ('-07') as its reference point throughout
YYYY_MM = [(str(year) + '-07') for year in range(1992, 2024)]
# Neat little trick to write to an IO stream in order to run the code here
with io.StringIO() as f:
    w = csv.writer(f)
    w.writerow(['iso_code', 'year', 'hot_days'])
    for iso_code in COUNTRY_ISO_CODES:
        # The API request encodes the exact data to extract,
        # with only the iso_code needing to be modified.
        climate_link = f'https://cckpapi.worldbank.org/api/v1/era5-x0.
 ↪25_timeseries_hd30_timeseries_annual_1950-2023_mean_historical_era5_x0.
 ↪25_mean/{iso_code}?_format=json'

        json_data = json.loads(requests.get(climate_link).text)
        extract = {
            year: hot_days for year, hot_days
            in json_data['data'][iso_code].items()
            if year in YYYY_MM
        }
        # Truncate the year back to just a YYYY format
        extract_with_iso_code = [
            (iso_code, year[:4], hot_days) for year, hot_days in extract.items()
        ]
        w.writerows(extract_with_iso_code)
    # Extra stuff due to using StringIO
    clim_file = f.getvalue()

# Show the raw JSON output for one of the countries and an extract of the␣
 ↪output file
print(json.dumps(json_data))
for line in clim_file.split()[:10]:
    print(line)
```

{"metadata": {"apiVersion": "v1", "status": "success", "messages": []}, "data":
{"USA": {"1950-07": 37.83, "1951-07": 46.22, "1952-07": 54.04, "1953-07": 53.84,
"1954-07": 54.51, "1955-07": 52.79, "1956-07": 52.05, "1957-07": 42.63,
"1958-07": 46.34, "1959-07": 51.07, "1960-07": 49.83, "1961-07": 46.08,
"1962-07": 48.27, "1963-07": 50.26, "1964-07": 48.74, "1965-07": 40.55,
"1966-07": 45.46, "1967-07": 40.06, "1968-07": 42.32, "1969-07": 45.77,
"1970-07": 51.27, "1971-07": 45.89, "1972-07": 44.33, "1973-07": 45.5,
"1974-07": 44.56, "1975-07": 43.9, "1976-07": 39.83, "1977-07": 49.43,
"1978-07": 48.75, "1979-07": 42.1, "1980-07": 53.8, "1981-07": 45.29, "1982-07":

```
41, "1983-07": 49.24, "1984-07": 45.86, "1985-07": 44.76, "1986-07": 46.36,
"1987-07": 49.04, "1988-07": 57.52, "1989-07": 47.29, "1990-07": 50.47,
"1991-07": 49.95, "1992-07": 38.33, "1993-07": 43.27, "1994-07": 48.32,
"1995-07": 47.53, "1996-07": 44.79, "1997-07": 41.16, "1998-07": 51.98,
"1999-07": 45.98, "2000-07": 52.24, "2001-07": 49.31, "2002-07": 55.19,
"2003-07": 48.68, "2004-07": 41.03, "2005-07": 55.46, "2006-07": 56.36,
"2007-07": 56.82, "2008-07": 48.54, "2009-07": 45.17, "2010-07": 56.91,
"2011-07": 59.97, "2012-07": 63.12, "2013-07": 51.43, "2014-07": 48.51,
"2015-07": 52.44, "2016-07": 57.69, "2017-07": 55.2, "2018-07": 60.6, "2019-07":
56.07, "2020-07": 61.05, "2021-07": 59.33, "2022-07": 62.87, "2023-07": 55.71}}}
iso_code,year,hot_days
AUT,1992,6.95
AUT,1993,2.23
AUT,1994,6.64
AUT,1995,2.41
AUT,1996,0.5
AUT,1997,0.32
AUT,1998,4.12
AUT,1999,1.17
AUT,2000,4.58
```

The site for the climate data has rules that limit how much data you can request at once depending on the size of your request. Because of this, we need to ask the API for one country at a time. Thankfully though, rate limiting was never an issue I had to worry about for something of this size.

It wasn't an important consideration for a project of this scale to care about the "indexing" month of the dataset being July. And while I doubt this is the reason why, I treat it as if July is the month after which everything resets, as the months leading up to and in July tend to be the warmest in the northern hemisphere. The fact that the annual data didn't neatly map 1:1 onto the calendar year was not of great concern, although it could be under different circumstances.

In any case, our files are now created and we are ready to start building and manipulating the dataframes that we will use.

2.5 Opbygning af dataframes

[6]:
```
# Allow Polars to infer all datatypes
# While you could get fancy and specify a schema with fx enums or force the␣
 ↪years
# into a date format, for the purposes of this project, it's not worth the␣
 ↪effort
pop = pl.read_csv('../data/population-processed.csv')
climate = pl.read_csv('../data/climate-data.csv')
tourists_pivoted = pl.read_csv('../data/tourists-processed.csv')
```

[7]:
```
# "Unpivot" the dataframe so that year repeats for each iso_code.
# This operation is more intuitive if you start with a pivoted table,
# as I originally did before realizing it was unnecessary.
tourists = tourists_pivoted.unpivot(
```

```
    index='iso_code', variable_name='year', value_name='visits'
)
```

[8]:
```
# We need to force cast the year to an int after pivoting to allow for joins␣
 ↪later on.
# It got interpreted as a string due to occuring on the same row as 'iso_code'␣
 ↪in the csv.
_int_year = pl.Series('year', tourists.select(pl.col('year').cast(pl.Int64)))
tourists.replace_column(tourists.get_column_index('year'), _int_year)
```

[8]:
```
shape: (578, 3)

 iso_code   year   visits
 ---        ---    ---
 str        i64    i64

 BEL        1992   643
 FIN        1992   177317
 FRA        1992   138251
 NLD        1992   1275478
 IRL        1992   0
 …          …      …
 SWE        2025   1487736
 DEU        2025   21494576
 AUT        2025   204271
 CAN        2025   129123
 USA        2025   1235272
```

[9]:
```
# Constants
COUNTRY_ISO_CODES = sorted(
    pop.unique(subset='iso_code').get_column('iso_code').to_list()
)

# I chose to exclude anyone under 15, as the relevant traveling population is in
# these two age ranges, and they likely exhibit different patterns between them.
# The total population is still needed to compare against however, as only␣
 ↪knowing
# the working and retired populations doesn't account for the younger␣
 ↪generations
# that will replace the people who age out.
WORKING = 'Y15T64'  # 15 to 64 years old
RETIRED = 'Y_GE65'  # 65+ years old
TOTAL   = '_T'      # Total population, including those under 15

# From find start year script
START_YEAR = 2005
```

```
# The canon column order
COLUMN_ORDER = [
    'iso_code',
    'year',
    'country',
    'hot_days',
    'visits',
    'demo',
    'demo_total',
    'country_total',
    'pop_percent',
]
```

[10]:
```
# Use explicit type hint for code completion
df_by_country: dict[str, pl.DataFrame] = {}

# TODO: better name?
# This dataframe holds the starting
# data for all further data analysis
monster_df: dict[str, pl.DataFrame] = {}

for iso_code in COUNTRY_ISO_CODES:
    # Get a scalar of the total population, rather than a series (.first())
    _pop_total_expr = pl.col('demo_total').filter(pl.col('demo') == TOTAL).
 ↪first()

    # .over() allows us to group_by and left join the result in the same␣
 ↪operation.
    # The call to .over() below expands (broadcasts) pop_total_expr to fill the␣
 ↪rows
    # for that particular iso_code and year. E.g. AUT 1992 will have 3 identical
    # entries for country_total, once each for the WORKING, RETIRED, and TOTAL␣
 ↪demos.
    _pop_with_stats = (
        pop.filter(pl.col('iso_code') == iso_code)
            .with_columns(
                _pop_total_expr                 # Grab the TOTAL population␣
 ↪(scalar) for
                    .over(['iso_code', 'year']) # that code+year combo, and␣
 ↪fill the rows
                    .alias('country_total')     # of country_total for the same␣
 ↪code+year
            ).with_columns( # Need a second call, as country_total must first␣
 ↪exist
                (pl.col('demo_total') / pl.col('country_total') * 100)
```

```
                    .round(2)
                    .alias('pop_percent')
            ).sort(pl.col('year'))
    )

    # Start with the tourists df, as it has all years (1992-2025). Grab the same country's
    # rows from climate, and do a join that resembles the .over() above. The difference
    # here is that there is a 1:1 correspondence between tourists and climate. The latter
    # is missing data for 2024 and 2025, so there will be some null values there. The
    # join with pop_with_stats however has multiple rows for each code+year, hence 1:m.
    # Joining on those two columns will automatically broadcast the appropriate rows.
    _tour = tourists.filter(pl.col('iso_code') == iso_code)
    _clim = climate.filter(pl.col('iso_code') == iso_code)
    _full_df = (
        _tour.join(_clim, on=['iso_code', 'year'], how='left', validate='1:1')
            .join(_pop_with_stats, on=['iso_code', 'year'], how='left', validate='1:m')
    )
    # The one null value we want to fill is the missing 'country' for 2025.
    # The other null values should stay null for now.
    _fixed_countries = pl.Series(
        'country',
        _full_df.select(pl.col('country').fill_null(strategy='forward')),
    )
    _full_df.replace_column(_full_df.get_column_index('country'), _fixed_countries)
    _full_df = _full_df.select(COLUMN_ORDER)

    # I am willfully choosing the duplication here for the reasons described above
    df_by_country[iso_code] = _full_df
    monster_df[iso_code] = _full_df.filter(pl.col('year') >= START_YEAR)
```

2.6 Datanormalisering

[11]:
```
# Define a metric for determining the starting year for our analysis, as there are
# a lot of very low outliers in the first several years of visits from multiple
# countries. One quick way for this is finding the earliest year where the minimum
```

```
# number of visits is at least equal to some multiple of the STD below the mean,
# as the data can be quite variable due to the initial outliers. We don't need␣
 ↪to
# worry about accidentally removing the COVID years, as this filter is only used
# to calculate the starting year. A couple countries (e.g. Iceland and Portugal)
# are so extreme that their starting STD is larger than the mean.
start_year = 1992
stable = False
count = 0
while not stable:
    stable = True # Assume stable until proven otherwise
    print('start_year', start_year)
    print('count', count)
    for iso_code in COUNTRY_ISO_CODES:
        visits = pl.col('visits')
        # Polars makes it very efficient to perform queries like this,
        # as it uses lazy execution under the hood and can plan ahead
        # for operations by taking advantage of predicate and projection
        # pushdown. For the code below, that means there is very little
        # sacrifice to performance by repeating the same year calculation
        # each loop, as the number of actual lookups will be minimal.
        # So instead of having to consider whether to copy a filtered
        # df_by_country every loop and sacrifice memory for more performance
        # in future iterations, you can naively repeat the same operations
        # on the full dataframe, at least in this case.
        year = df_by_country[iso_code].filter(
            # First filter out the years, as applying both filters
            # concurrently defeats the purpose of the second one.
            pl.col('year') >= start_year
        ).filter(
            # 1.2 is a "magic number" - I picked it more so that the
            # start_year would end somewhere reasonable, but a more
            # systematic and reasoned approach would likely be needed
            # under different circumstances. 2005 ends up being the
            # start_year, as the last outliers from 2004 - Iceland and
            # Portugal - both jump by an order of magnitude from 2004
            # to 2005, and after that the changes are smoother.
            visits >= (visits.mean() - (visits.std() * 1.2))
        ).select(
            pl.min('year')
        ).item()
        if year > start_year:
            start_year = year
            stable = False
            count += 1
            print(iso_code)
            print(year)
```

```
            print('count:', count)
```

```
start_year 1992
count 0
AUT
1996
count: 1
ISL
2000
count: 2
start_year 2000
count 2
BEL
2001
count: 3
ISL
2005
count: 4
start_year 2005
count 4
```

[12]:
```
# We only need this list to concat all the dataframes together,
# so there's no need for a dict like I have been using.
_scaled_by_pop = []
for iso_code in COUNTRY_ISO_CODES:
    country = (
        monster_df[iso_code]
            .filter(pl.col('demo') == TOTAL)
            .select(
                # Keep the country names so we can use them in graphs and plots
                pl.col('iso_code', 'year', 'country'),
                scaled=((pl.col('visits') / pl.col('demo_total')) * 100).
 ↪round(4))
    )
    _scaled_by_pop.append(country)
```

[13]:
```
# In order to render the years properly in Altair, we need them as strings
scaled_df = (
    pl.concat(_scaled_by_pop)
        .with_columns(pl.col('year').cast(pl.String).name.prefix('str_'))
)
```

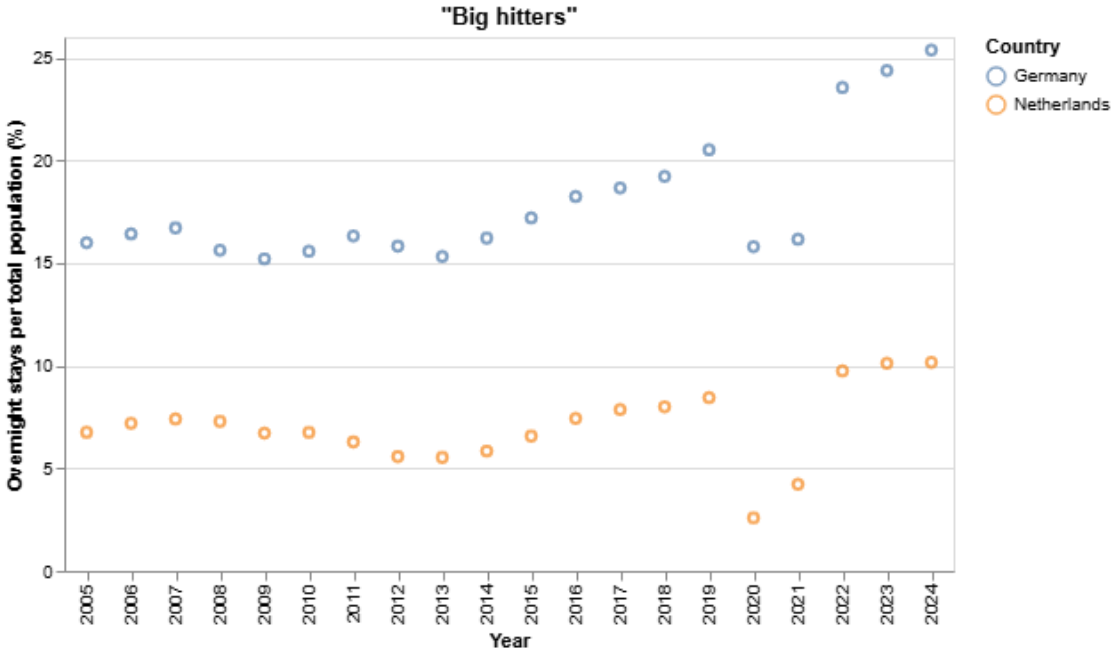
2.7 Visualisering

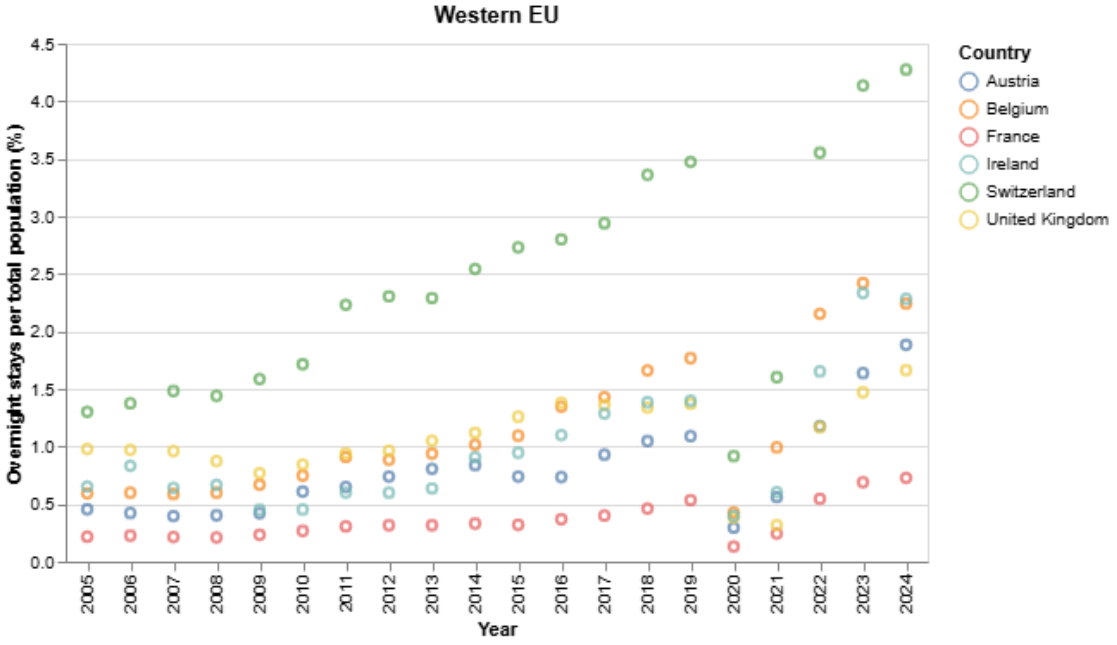
[14]:
```
# Constants
# TODO better name for big hitters
BIG_HITTERS = ['DEU', 'NLD']
```

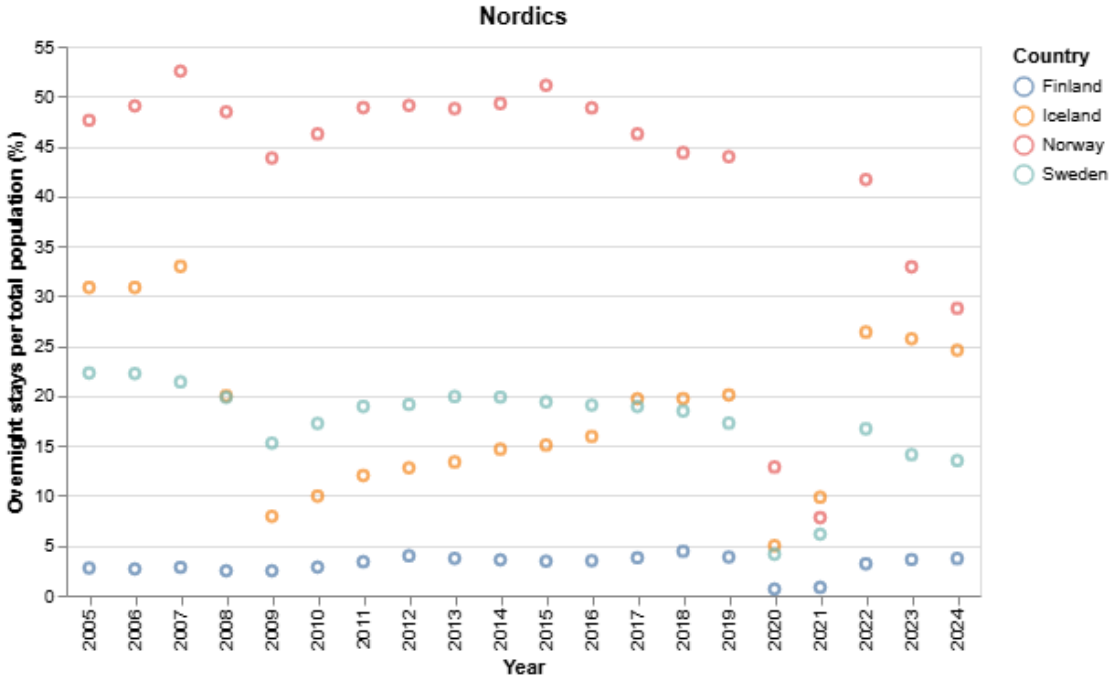
```
WEST_EU     = ['AUT', 'BEL', 'CHE', 'FRA', 'GBR', 'IRL']
NORDICS     = ['FIN', 'ISL', 'NOR', 'SWE']
SOUTH_EU    = ['ESP', 'ITA', 'PRT']
NOR_AMER    = ['CAN', 'USA']

REGIONS = {
    'big_hitters': BIG_HITTERS,
    'west_eu': WEST_EU,
    'nordics': NORDICS,
    'south_eu': SOUTH_EU,
    'nor_amer': NOR_AMER,
}
```

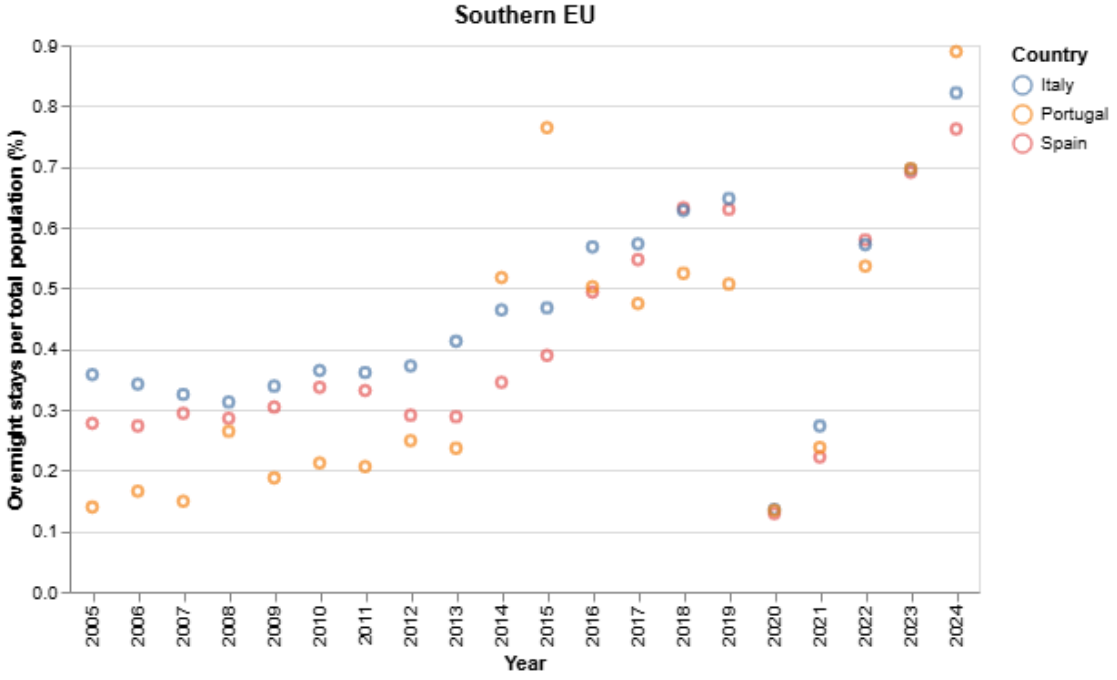
[15]:
```
regions_df = {}
for region in REGIONS.keys():
    regions_df[region] = scaled_df.filter(pl.col('iso_code').
 ↪is_in(REGIONS[region]))
```

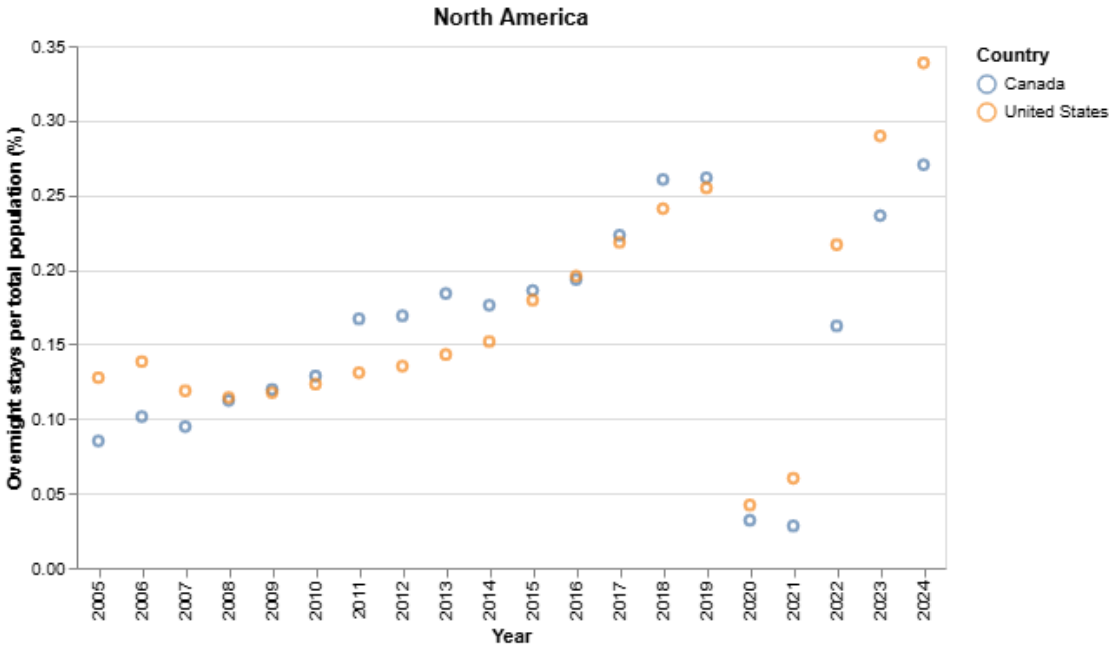
[16]:
```
import altair as alt

REGION_TITLES = {
    'big_hitters': '"Big hitters"',
    'west_eu': 'Western EU',
    'nordics': 'Nordics',
    'south_eu': 'Southern EU',
    'nor_amer': 'North America',
}

charts = {}
for region, df in regions_df.items():
    charts[region] = (
        alt.Chart(df).mark_point(tooltip=True).encode(
            x=alt.X('str_year', title='Year'),
            y=alt.Y('scaled', title='Overnight stays per total population (%)'),
            color=alt.Color('country', title='Country'),

        )
        .properties(
            width=500,
            title=REGION_TITLES[region],
        )
        .configure_scale(zero=True)
    )
```


"Big hitters"
Overnight stays per total population (%)
0
5
10
15
20
25
2005
2006
2007
2008
2009
2010
2011
2012
2013
2014
2015
2016
2017
2018
2019
2020
2021
2022
2023
2024
Year
Country
Germany
Netherlands


Western EU
Overnight stays per total population (%)
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0
4.5
2005
2006
2007
2008
2009
2010
2011
2012
2013
2014
2015
2016
2017
2018
2019
2020
2021
2022
2023
2024
Year
Country
Austria
Belgium
France
Ireland
Switzerland
United Kingdom


Nordics
Overnight stays per total population (%)
Year
Country
Finland
Iceland
Norway
Sweden


Southern EU
Overnight stays per total population (%)
Year
Country
Italy
Portugal
Spain


North America
Overnight stays per total population (%)
0.35
0.30
0.25
0.20
0.15
0.10
0.05
0.00
2005
2006
2007
2008
2009
2010
2011
2012
2013
2014
2015
2016
2017
2018
2019
2020
2021
2022
2023
2024
Year
Country
Canada
United States